2026 소개딩 SW개발보안 경진대회 | 트랙B 기술혁신

LogosFuzz: 빌드 구조와 API 제약 분석 기반 퍼징 하네스 자동 생성 보안 테스트 시스템

C/C++ 오픈소스 · 빌드 구조와 API 제약을 분석해 지식 추출부터 크래시 판별까지 자동화

김민주, 송서원, 임세은, 황다연 (서울여자대학교) | 팀 Fuzzing Candy

3,080
S-CORE C++ API 추출 · 682개 로직 그룹

181→216
S-CORE 커버리지 · 동일 조건, 300초 실행

2 / 2
LLM 생성 하네스 빌드 성공 · 자가치유 1회

1,182
단위·통합 테스트 통과

프로젝트 개요

1. 개발 배경

- **오픈소스 공급망 보안 위험**: 대부분의 SW가 C/C++ 오픈소스 라이브러리를 재사용하므로, 라이브러리 하나의 결함이 의존하는 SW 전체로 확산
- **메모리 안전성 결함**: 길이, 할당 크기, 객체 수명 같은 경계 조건은 정상 기능 시험으로 놓치기 쉬움
- **퍼징의 높은 진입장벽**: OSS-Fuzz는 1,000개 프로젝트에서 10,000건 이상의 취약점을 발견했지만, 수작업 하네스와 복잡한 빌드 설정으로 일반 조직의 도입이 어려움
- **자동차와 임베디드의 시급성**: SDV의 진단, 로깅, 통신이 C/C++ 오픈소스에 의존하며 UN R155(CSMS) 규제 준수가 의무화됨

주요 사례	내용	KISA 보안약점
Heartbleed (2014)	메모리 경계 검증 누락 → 대규모 정보 유출	메모리 버퍼 오버플로우
BadAlloc (2021)	할당 크기 오류 → 임베디드 SDK 등 CVE 25개 이상	정수형 오버플로우
Redis CVE-2025-49844	조작된 Lua 스크립트의 use-after-free → 원격 코드 실행	해제된 자원 사용

2. 기존 방식의 한계

- **하네스와 빌드 수작업**: 프로젝트당 수 시간의 수작업과 고도의 전문지식 필요
- **C++와 Bazel 구조 복원 실패**: score/json 기준 106개 파일이 있어도 API 0개 추출
- **환경 의존 오류**: 하드웨어와 외부 함수 부재로 링크 실패와 가짜 크래시 발생

3. LogosFuzz 개발 목표

"대상 프로필 1개 설정, 명령어 한 줄로 지식 추출부터 크래시 판별까지 자동 수행"

```
$ logosfuzz run --profile targets/libsndfile.json
  --budget 600 -o out/libsndfile
```

- 소스, 주석, 빌드 정보에서 API 제약을 추출해 지식베이스 구축
- 연관 API를 상태 기반 로직 그룹으로 묶어 하네스 생성
- 컴파일/링크 오류 분류 기반 자가치유, 누락된 HAL 함수 자동 Mock 주입
- Docker 격리 퍼징, ASan/UBSan 메모리 오류 탐지
- 규칙 기반 판별과 사람 승인을 거쳐 오탐을 지식베이스에 역피드백

기존 도구 비교

비교 항목	Locus	Prome Fuzz	OSS-Fuzz-Gen	lib Erator	Logos Fuzz
LLM 기반 하네스 생성	△	○	○	×	○
API 문서 활용(RAG)	×	○	△	×	○
API 호출 순서 추론	△	△	×	○	○
컴파일/링크 오류 자동 수정	×	×	○	×	○
Coverage 기반 최적화	○	○	○	△	○
크래시 정탐/오탐 자동 분류	×	○	×	×	○
Bazel, 임베디드 C/C++ 대응	×	×	×	×	○

○ 지원 △ 일부 지원 × 미지원

시스템 설계

1. 시스템 구성

입력	오픈소스 소스코드, 대상 프로필(targets/*.json), 빌드 어댑터(Bazel, CMake, prebuilt)
AI와 지식 계층	LLM(GPT-4o) 하네스 생성과 퍼징 오류 자가치유, 지식베이스 + BM25 RAG, 규칙 기반 크래시 판별 + 예외 케이스 LLM 보완
코어 엔진	정적 분석(Bazel query, compile_commands, libclang AST), 동적 퍼징(libFuzzer), 오류 탐지와 계측(ASan, UBSan, llvm-cov)
보안 경계	Docker 격리 실행(네트워크 차단, 권한 최소화, 자원과 파일 보호), HITL 승인 게이트(지식베이스 변경과 취약점 공개는 사람 승인 후 반영)

2. 5단계 파이프라인

EXT 지식 추출	SCH 스케줄링	GEN 하네스 생성	EXE 퍼징 실행	ANA 크래시 분석
Bazel query, AST와 주석에서 API 제약 KB 생성	연관 API 로직 그룹화, 시너지 점수와 시간 배분	LLM 하네스 생성, 오류 분류 자가치유	Docker 격리 libFuzzer, ASan/UBSan, 커버리지	중복 제거, 규칙 판별, 리포트, KB 역피드백

오탐 역피드백: ANA 오탐 원인 분석 → 지식베이스 변경안(diff) → 사람 승인 → KB 반영 → 해당 그룹 하네스 재생성

3. 핵심 기술

기술	설명
다중 소스 지식 추출	AST, 주석, 분기문에서 NULL 검사, 버퍼-길이 쌍, 값 범위, 해제 책임을 추출해 신뢰도와 함께 저장하고 하네스 프롬프트에 주입
빌드 자가치유	컴파일 오류와 3대 링커 오류(미정의 심볼, 중복 정의, 라이브러리 누락)를 분류해 LLM으로 수정하고, 미정의 HAL 함수는 weak 스텁 자동 생성
크래시 판별과 역피드백	스택 서명 중복 제거와 함수별 에러 반환 규격 기반 규칙 판별, 신뢰도 0.8 미만은 사람 검토와 증거 주입 LLM 판별로 정확도 **10% → 85%** (dlt-daemon 크래시 20건 기준)

보안약점 예방 설계

1. 설계 원칙: 외부 코드, LLM 생성 코드, 크래시 입력은 신뢰하지 않음

원칙	적용 내용
신뢰 경계 분리	LLM 생성 코드와 퍼징 입력은 격리 컨테이너에서만 실행 (--network none, cap-drop ALL, 자원 상한, 읽기 전용 마운트)
명령 문자열 미사용	외부 도구는 인자 리스트로만 호출 (shell=True, os.system 0건), LLM 출력은 eval/exec 없이 C 코드로만 컴파일
사람 승인 게이트	KB 변경과 취약점 공개는 HITL 승인 필수, 락과 원자적 쓰기의 append-only 감사 로그
비밀정보와 출력 보호	API 키는 환경변수로만 수신, HTML 리포트의 모든 문자열 esc() 처리

2. 참고 가이드라인과 적용

가이드라인	주요 적용 내용
소프트웨어 개발보안 가이드 (행안부, KISA 2021)	구현단계 49개 항목 적용표 작성, 경로 조작(safe_join)과 명령어 삽입(argv 호출) 차단
SW 보안약점 진단가이드 (행안부, KISA 2021)	Bandit, pip-audit, Gitleaks 점검, 보안 회귀 테스트 53개로 재검사
Python 시큐어코딩 가이드 (과기정통부, KISA 2023)	eval/exec, pickle 미사용, 비밀 환경변수 차단, 오류 메시지 절대경로 제거
AI 보안 안내서 (과기정통부, KISA 2025)	LLM 전송 전 비밀값 마스킹, 비신뢰 데이터 분리, KB 변경과 CVE 공개는 사람 승인
SW 공급망 보안 강화 로드맵 (관계부처 합동 2026)	도입 전 오픈소스 보안 검증 자동화, 이미지 digest와 의존성 버전 고정

범용 대상 프로필

대상마다 **JSON 프로필 하나**만 작성하면 빌드 방식이 달라도 같은 명령어로 실행

범용 C libsndfile 1.2.2	자동차 Eclipse S-CORE	임베디드 HAL 샘플
빌드: CMake 에러 규격: C 반환코드 API 8개 지정, 누락 0	빌드: Bazel 에러 규격: score::Result C++ 클래스와 템플릿 API 추출	빌드: prebuilt 미정의 HAL 함수 weak 스텁 자동 생성, 하드웨어 없이 퍼징

회귀 테스트 자동화: 범용화 후 빌드 요약과 BUILD.bazel 결과물을 범용화 전 기준 결과와 pytest로 교차 검증

성과 및 검증

1. 주요 성과 (단위/통합 테스트 1,182개 통과)

3,080
S-CORE C++ API 추출 (기존 0개, 그룹 682개)

216
S-CORE 커버리지 (기존 하네스 181, 300초)

86.4%
임베디드 라인 커버리지 (HAL 스텁 후 힙 OOB 탐지)

2. 대상별 검증 결과

대상	빌드	검증 결과
Eclipse S-CORE (자동차)	Bazel	API 3,080개, 그룹 682개, 300초 커버리지 216 (기존 181)
COVESA DLT (자동차)	compile_commands	API 914개, 제약조건 보유 80.6%, 초당 373,855회 실행
libsndfile 1.2.2 (범용 C)	CMake	전 단계 연결 검증, UBSan 정수 오버플로우 고유 6건 (검토 대기)
임베디드 HAL 샘플	prebuilt	HAL 함수 2개 자동 스텁, 힙 OOB 탐지, 라인 86.4%, 브랜치 70.0%

※ libsndfile·임베디드는 수동 하네스로 전 단계를 검증, S-CORE·DLT는 LLM 생성 하네스 실측

3. 자체 보안 점검 (Bandit, pip-audit, Gitleaks)

47→29건
Bandit 검출 (High, Medium 0건)

100%
조치 대상 18건 모두 조치

0건
의존성 취약점, 비밀정보 노출

조치 내역: SHA1, MD5 → SHA-256(4건), 표준 XML 파서 → defusedxml(2건), except-pass → 로그 기록(10건), assert와 /tmp 고정 경로 제거(2건). 남은 29건은 셸 없이 인자 리스트로 호출하는 subprocess 안내성 경고

결론

1. 기대효과

- **하네스 작성 자동화**: 프로필 1회 작성으로 하네스 수작업을 대체하고 반복 빌드 오류 수정 시간 단축
- **탐지 범위 확대**: 자동 스텁으로 하드웨어 없이 임베디드 코드를 탐색하고, 에러계약 판별로 실제 취약점 후보에 집중
- **오픈소스 공급망 검증**: 도입 전 보안 검증 자동화, UN R155 테스트 증거 축적

2. 향후 계획

- **기술 고도화**: ASan 크래시 위치 복원 후처리, 생성 함수 누락 로직 그룹 보완
- **검증 자동화**: GitHub Actions 연동 CI 보안 검증, 생성 하네스 기반 실측 대상 확대
- **적용 범위 확대**: 바이너리 대상 퍼징, 다양한 CPU 아키텍처 지원